

Informe de Arquitectura de la Aplicación Generador de Ramales

Documento técnico de la arquitectura **frontend** (interfaz web Streamlit) y **backend** (pipeline geoespacial determinista en Python), con el flujo de datos extremo a extremo. Genera 4 trazados diferenciados sobre GIS público y los compara en una tabla multicriterio.

ÍNDICE

1. Visión general y modelo de ejecución
2. Frontend y Backend
3. Mapa de componentes
4. Frontend — interfaz Streamlit
5. Backend — pipeline geoespacial
6. Fuentes de datos y reproducibilidad
7. Modelo de coste y ponderación
8. Estado del código y stack

01 Visión general y modelo de ejecución

La aplicación combina una **interfaz web** y un **pipeline geoespacial por lotes** que comparten el mismo proceso Python. La interfaz importa e invoca directamente las funciones del pipeline; el intercambio de datos se hace por **archivos en disco** (GeoTIFF, GeoPackage, YAML) y por el `st.session_state` de Streamlit.

El **frontend** es un único archivo, `src/app/streamlit_app.py`, que se arranca con `streamlit run`: un asistente de 3 pasos con mapas Folium, editor de pesos y página de resultados. El **backend** son los módulos deterministas de `src/` (`ingesta · superficie · trazados · métricas`), que descargan y alinean las capas GIS, construyen las superficies de coste por perfil, calculan el camino de mínimo coste (LCP) y reúnen las métricas multicriterio.

Principio de diseño rector: *alinear antes de combinar, diferenciar antes de comparar*. Ninguna capa entra en el cálculo sin estar reproyectada a **EPSG:25830** y remuestreada a la **rejilla común**. El coste es siempre un **índice adimensional**, nunca un valor económico.

Cómo se ejecuta el pipeline

La vía principal y operativa es la interfaz web (streamlit_app.py → _ejecutar_pipeline()), que actúa como orquestador real encadenando preparación, trazado y métricas; de forma **parcial** también puede ejecutarse etapa a etapa por CLI de módulo (python -m src.trazados.ruta_pendiente / src.mtricas.calculo), donde cada etapa tiene su propio main() ejecutable y reproducible.

Estructura de carpetas del proyecto

```
proyecto/
├── src/
│   ├── ingesta/      # descarga, recorte, reproyección y alineación de capas GIS
│   ├── superficie/   # superficies de coste por criterio (raster [0,1]) + combinación por perfil
│   ├── trazados/     # motor LCP (Dijkstra 8-conexo) + diferenciación de rutas
│   ├── metricas/     # métricas multicriterio por ruta + diversidad de corredores
│   ├── comparacion/  # comparador y scoring formal (esqueleto)
│   └── app/          # streamlit_app.py – interfaz web (orquestador puente) + informe.py (PDF)
├── data/
│   ├── config/       # escenario.yaml (AOI, origen/destino) · perfiles.yaml (pesos) – versionados
│   ├── raw/          # capas GIS originales descargadas o aportadas a mano – NO versionadas
│   └── processed/    # rasters alineados, capas de coste, superficies y rutas – NO versionadas
├── tests/            # pruebas (alineación de rasters, métricas)
└── notebooks/       # exploración geoespacial y ejercicios de formación
```

02 Frontend y Backend — separación de responsabilidades

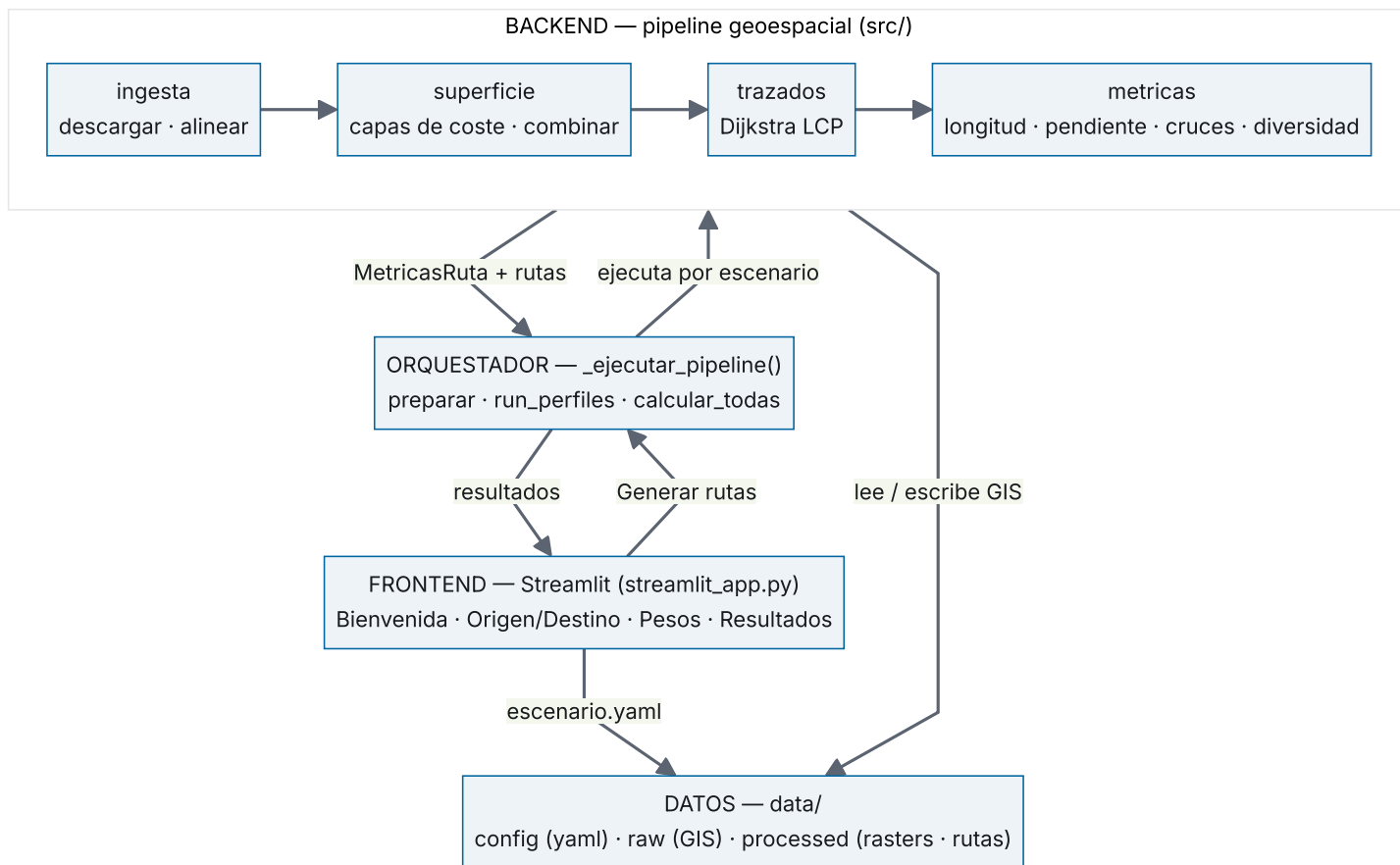
Referencia rápida de **qué incluye cada capa**: archivos, funciones, entradas, salidas, tecnologías y estado, más la costura que las une.

	FRONTEND	BACKEND
Archivos	src/app/streamlit_app.py (único archivo).	src/ingesta/ · src/superficie/ · src/trazados/ · src/metricas/
Funciones clave	_main, _render_paso1/2, _render_results, _render_editor_pesos, _mapa_entrada/_resultados, _ejecutar_pipeline (orquestador puente).	descargar_capas, alinear_capas, preparar, combinar_pesos, run_perfiles, dijkstra, calcular_todas.
Entrada	Clics e inputs del usuario; lee escenario.yaml y perfiles.yaml.	Capas GIS de data/raw/; pesos por perfil; origen/destino en UTM.
Salida	HTML/CSS + mapas Folium en el navegador; escribe coordenadas a escenario.yaml.	GeoTIFF (rasters, superficies), GeoPackage (rutas), dict MetricasRuta.
Puente	_ejecutar_pipeline() es la única costura entre capas: importa el backend en caliente (tras purgar sys.modules), lo invoca por escenario y traduce el progreso a la barra de Streamlit. Sustituible por la CLI (src/app/cli.py, pendiente) sin tocar el backend.	

Implicación para el futuro: como el backend no depende de la interfaz, se puede envolver en una API REST, un job por lotes o un notebook sin reescribir el pipeline. El único acoplamiento a Streamlit vive en streamlit_app.py; todo src/ por debajo es reutilizable tal cual.

03 Mapa de componentes

Arquitectura en cuatro capas con comunicación descendente. La interfaz solo habla con el motor a través del orquestador _ejecutar_pipeline(); el backend y la interfaz intercambian datos leyendo y escribiendo archivos en data/. El diagrama sigue el flujo real: la petición baja de Frontend a Backend y los resultados suben de vuelta.



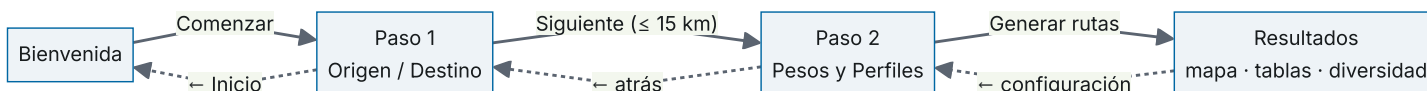
Lectura del diagrama: la columna central es el recorrido de una petición (Frontend → Orquestador → Backend → Orquestador → Frontend); las dos flechas hacia DATOS representan toda la persistencia en disco (config, capas descargadas y salidas del pipeline). El orden exacto de las llamadas por escenario se detalla en el backend (orquestración de `_ejecutar_pipeline()`).

04 Frontend — interfaz Streamlit

Toda la interfaz vive en `src/app/streamlit_app.py`. Es una **single-page app** que cambia de «pantalla» según el valor de `st.session_state.pantalla`. El punto de entrada es la función `_main()`, invocada al final del módulo, que pinta el *stepper* de progreso y la pantalla activa.

Navegación de la interfaz

La app cambia de pantalla según `st.session_state.pantalla`. El avance (línea continua) valida en cada paso; la vuelta atrás (línea punteada) no recalcula nada.



Paso 1 · Origen y Destino `_render_paso1`

- Gestión de escenarios: crear, renombrar y borrar (A, B, ...). Los escenarios «de fábrica» se restauran al abrir cada sesión; los añadidos en sesión son efímeros y sus salidas se limpian solas al cerrarla.
- Entradas numéricas X/Y en **EPSG:25830** (metros) para origen y destino.
- Mapa Folium con ortofoto Esri; un clic fija el punto activo (origen/destino) — convierte lat/lon → UTM con `pyproj.Transformer`.
- Valida la distancia máxima `MAX_DIST_M = 15 km`; bloquea «Siguiente» si se supera.
- Tarjeta «Datos para esta zona»: estado real por capa contra disco (descargada / pendiente / manual). Para Catastro, localiza los **municipios del corredor** (límites IGN), enlaza a la Sede Catastro INSPIRE y acepta el **ZIP subido desde la propia interfaz** (GML INSPIRE, Shapefile de la Sede o paquete provincial), que se convierte y recorta automáticamente.
- Al avanzar, persiste las coordenadas en `data/config/escenario.yaml`.

Paso 2 · Pesos y Perfiles `_render_paso2`

- Editor de pesos por capa (sliders 0–100 %) para los 4 perfiles: corto, equilibrio, ambiental, pendiente.
- 7 capas ponderables: longitud, TPI (relieve), protegida, inundable, cruces, expropiación, geotecnia.
- Control de la **barrera dura de pendiente** (70 % por defecto, editable): pendiente máxima transitable; el pipeline regenera la capa TPI cuando el umbral pedido difiere del grabado en el GeoTIFF.
- «Generar rutas» dispara `_ejecutar_pipeline()` con `perfiles_override` (los pesos editados en la UI, sin tocar el YAML).

Resultados `_render_results`

- **Tab Mapa:** `_mapa_resultados()` lee los `ruta_{s}_{perfil}.gpkg`, los reproyecta a WGS-84 y los dibuja como `FeatureGroup` por escenario con `LayerControl` y leyenda de perfiles.
- **Tabs Métricas (por escenario):** tabla de `MetricasRuta` (km, coste rel., pendiente, km protegida/inundable/urbano, nº cruces) + tarjetas KPI.
- **Tab Diversidad:** matriz de solapamiento dirigido entre corredores (para juzgar si las rutas son realmente distintas).
- **Informe PDF por escenario** (`src/app/informe.py`): metodología, ráster + ruta y comparativa por perfil, con caché invalidada por contenido; y **descarga ZIP** de las rutas (`.gpkg` + estilos `.qml`).
- Avisos de preparación: capas de coste que no se pudieron generar (y por tanto **no entraron** en el cálculo) o descargadas de forma **incompleta** (p. ej. una colección SNCZI caída).

05 Backend — pipeline geoespacial

El backend es una cadena de módulos deterministas. El orquestador `_ejecutar_pipeline()` (dentro del frontend) los encadena así, por cada escenario:

0 · Orquestación `streamlit_app._ejecutar_pipeline`

- Purga el caché de módulos de `sys.modules` (para recoger cambios de config en caliente) e importa `run_perfiles`, `calcular_todas`, `preparar`.
- Limpia las **salidas huérfanas** (rutas/superficies de escenarios que ya no existen) y, si el escenario **no está preparado** (faltan capas en `Capas_Coste/`) → llama a `preparar()` (descarga + alineación + superficies).
- Regenera la capa TPI cuando el **umbral de la barrera de pendiente** pedido en la UI difiere del grabado en `tpi_{s}.tif` (etiqueta del GeoTIFF).
- Traza las rutas (`run_perfiles`) y calcula las métricas (`calcular_todas`), reportando progreso a la barra de Streamlit vía `progress_cb`.

Etapas del pipeline

1 · Ingesta y alineación `src/ingesta/`

descargar_capas.py — descarga por bbox del AOI de cada escenario, vía servicios espaciales públicos:

CAPA	FUENTE / SERVICIO	DESCARGA	SALIDA
DEM (~30 m)	Copernicus GLO-30 (tiles COG en AWS S3)	automática	DEM_{s}.tif
Viario + ferrocarril	OpenStreetMap (Overpass API, 3 endpoints con reintento)	automática	OSM_{s}.gpkg
Hidrografía	WFS IGN INSPIRE (hy-n:WatercourseLink)	automática	HID_{s}.gpkg
Geología (litología)	IGME MAGNA50 (ArcGIS REST)	automática	IGME_{s}.gpkg
Zonas inundables	SNCZI MITECO (OGC API Features, vector ; unión T10+T100+T500)	automática	INUND_{s}.gpkg
Red Natura 2000	MITECO (OGC API Features, biodiversidad:RedNatura)	automática	RN2000_{s}.gpkg
Catastro (uso del suelo)	Sede Catastro INSPIRE — ZIP por municipio subido desde la app; foral (Navarra/P. Vasco) por bbox	manual asistida	Catastro/...

Qué se descarga solo y qué hay que aportar a mano:

- **Todas las capas GIS se descargan automáticamente salvo el Catastro nacional** (no hay servicio bbox fiable). La aportación es **asistida desde la app**: la tarjeta de Catastro del Paso 1 localiza los municipios del corredor, enlaza a la Sede Catastro INSPIRE y convierte el ZIP subido (GML INSPIRE, Shapefile de la Sede o paquete provincial) en PARCELA.shp recortado al AOI. El catastro **foral** (Navarra y País Vasco) sí se descarga por bbox (catastro_foral.py).
- **Red Natura 2000 es automática pero opcional**: se descarga vía OGC API Features de MITECO; si el servicio falla, el pipeline **continúa con un aviso** y ese criterio no entra en el coste del escenario.
- **Zonas inundables (SNCZI)**: vector oficial con atributos vía OGC API Features de MITECO (colecciones T10/T100/T500). Si alguna colección falla habiendo datos de otras, la capa se marca **parcial** y llega como aviso a la pantalla de resultados.

Para las capas automáticas, escribe un **manifiesto de estado** (manifiesto_estado.json) con contrato ok / empty / failed / partial: failed **no** escribe un GPKG vacío y devuelve código $\neq 0$ para que el pipeline no avance a ciegas (distingue «sin cobertura confirmada» de «descarga fallida»); partial escribe los datos disponibles pero se degrada a **aviso visible** en resultados (capa posiblemente incompleta).

aligner_capas.py — construye el AOI, reproyecta todo a **EPSG:25830**, remuestrea rasters a la rejilla común, recorta/reproyecta vectores a GeoPackage, detecta solapamientos (umbral 70 %) y **deriva la capa de coste TPI** del DEM alineado.

2 · Superficies de coste src/superficie/

Un módulo por criterio convierte cada capa alineada en un **coste por celda** estandarizado en $[0, 1]$, y lo deposita en Capas_Coste/{capa}_{s}.tif:

- geotecnia.py — coste por litología (campo DLO del IGME).
- expropiacion.py — coste por tipo catastral (urbano / rústico / dominio público).
- zonas_protegidas.py — binaria dentro/fuera de Red Natura 2000.
- zonas_inundables.py — binaria dentro/fuera de lámina SNCZI.
- cruces_viario_rios.py — coste puntual de cruce (viario, ferrocarril, ríos).
- tpi.py — relieve: cresta barata / valle caro; e impone la **barrera dura de pendiente** (umbral y equivalencia en grados, en el §07 Modelo de coste).

Para más información (tablas de coste por celda de cada capa), consulta el Diagrama de Flujo, §02.

combinar.py → **combinar_pesos(s, pesos)** concentra **toda la ponderación en un único punto**: suma cada capa de coste multiplicada por su peso de perfil, más un término base de longitud para que la distancia siempre cuente (la fórmula exacta, en el diagrama de flujo §03).

Convenio de impasabilidad: NODATA (-9999) e $\inf \rightarrow \text{nan}$; una celda es intransitable si *cualquier* capa lo es (el nan se propaga por la suma).

3 · Motor de trazado (LCP) src/trazados/ruta_pendiente.py

run_perfiles(s, perfiles_override): por cada perfil obtiene la superficie ponderada, *snapea* origen y destino a la celda válida más cercana, y ejecuta **Dijkstra 8-conexo isótropo** (coste de transición = distancia $\times$ media de las dos celdas; la fórmula, en el diagrama §03).

Exporta cada ruta a `Rutas/ruta_{s}_{perfil}.gpkg` + estilo QGIS `.qml`, y guarda la superficie del perfil en `Trazados/superficie_{s}_{pid}.tif`.

Diferenciación de rutas: se logra por **perfiles con pesos distintos** → superficies de coste distintas → corredores distintos. La validación posterior se hace midiendo el solapamiento (métrica de diversidad).

4 · Métricas multicriterio `src/metricas/calculo.py`

`calcular_todas()` reúne, por cada ruta, las métricas parciales de módulos independientes en un `MetricasRuta` (dataclass). Un módulo que falla por falta de datos no aborta el cálculo: registra el error en `MetricasRuta.errores` y deja el campo a 0.

MÓDULO	MÉTRICA
<code>longitud.py</code>	<code>longitud_km · coste_relativo</code> (índice)
<code>pendiente.py</code>	<code>pendiente_max_pct · pendiente_media_pct</code> (Horn sobre el DEM)
<code>zonas_protegidas.py</code>	<code>km_protegida · pct_protegida</code>
<code>zonas_inundables.py</code>	<code>km_inundable · pct_inundable</code>
<code>zonas_urbanas.py</code>	km de suelo urbano / diseminado / rústico / especial (catastro)
<code>cruces.py</code>	nº cruces de ríos / carreteras / ferrocarril
<code>diversidad_corredores.py</code>	matriz de solapamiento dirigido entre corredores

La **diversidad** se calcula una vez por escenario: engorda cada ruta a un corredor (buffer 60 m) y mide qué % de la longitud de A discurre dentro del corredor de B (asimétrico).

Comparación y ranking: se construyen directamente en la página de resultados de Streamlit (DataFrames + KPIs + matriz de diversidad).

06 Fuentes de datos, trazabilidad y reproducibilidad

Esta sección reúne todo lo necesario para **reproducir** una ejecución, **auditar** de dónde sale cada número y **retomar o mejorar** el sistema en el futuro.

Cómo reproducir una ejecución

```
# 1. Entorno (desde proyecto/) – script oficial: crea proyecto/.venv (Python 3.10),
#   instala requirements.txt (pip freeze del entorno validado) y verifica los imports
.\setup.ps1                                # Windows PowerShell   (Linux/macOS: ./setup.sh)
python check_entorno.py                    # verificación OK/FALTA, útil tras cada git pull

# 2. Única capa manual: CATASTRO nacional. Se aporta desde la propia app:
#   tarjeta de Catastro del Paso 1 → enlaces por municipio a la Sede Catastro
#   INSPIRE → subir el ZIP (la app lo convierte y recorta sola).
#   (RN2000 y zonas inundables SNCZI se descargan solas, vía OGC API Features de MITECO)

# 3a. Interfaz web (recomendado):
streamlit run src/app/streamlit_app.py

# 3b. O por línea de comandos, etapa a etapa (mismo resultado, reproducible):
python -m src.ingesta.descargar_capas    --escenario A -y
python -m src.ingesta.alinear_capas      --escenario A
python -m src.ingesta.preparar_escenario --escenario A      # descarga+alinear+superficies
python -m src.trazados.ruta_pendiente   --escenario A      # rutas por perfil
python -m src.metricas.calculo           --escenario A      # métricas + diversidad
```

Determinismo: el pipeline es determinista — mismos datos de entrada + mismos pesos ⇒ mismas rutas. El único origen de variación es la **capa descargada** (los servicios GIS pueden actualizarse); por eso el manifiesto de estado y las capas aportadas a mano son clave para congelar una ejecución reproducible.

Trazabilidad: de la fuente al número final

Cada métrica se puede rastrear hacia atrás por esta cadena de artefactos en disco:

```
Fuente GIS → data/raw/{CAPA}_{s}          (descarga cruda + manifiesto_estado.json)
            → Rasters_AOI/ · Recorte_AOI/  (alineado a EPSG:25830 + rejilla común)
            → Capas_Coste/{capa}_{s}.tif   (coste/celda [0,1], estandarizado A/38)
            → Trazados/superficie_{s}_{perfil}.tif (Σ peso·capa, por perfil)
            → Rutas/ruta_{s}_{perfil}.gpkg (LCP Dijkstra) → MetricasRuta
```

Los pesos que produjeron cada superficie quedan registrados en `perfiles.yaml` (o, si se editaron en la UI, en `st.session_state.perfiles_procesados`, mostrados en la página de resultados en «Pesos de capas usados en este procesamiento»). Los estilos `.qml` acompañan a cada raster/ruta para inspección directa en QGIS.

07 Modelo de coste y ponderación

Toda capa se estandariza a $[0, 1]$ antes de combinarse, para que la suma ponderada sea comparable. Las capas con condicionante en la **metodología oficial de Enagás** derivan su coste del factor de ponderación oficial A con divisor común $A_{REF} = 38$ (el mayor factor, «terrenos inestables»):

$$\text{coste_celda} = A_{\text{oficial}} / 38$$

CONDICIONANTE	FACTOR OFICIAL (A)	COSTE ESTANDARIZADO
Red Natura 2000	28,5	0,75
Urbano alta densidad	25,25	0,66
Inundabilidad	14,25	0,375
Roca dura / río permanente	13	0,34
Yeso / arcilla expansiva (barrera de mayor coste)	38	1,00

- El **TPI (relieve)** queda fuera del esquema A/38 (no es un condicionante de la tabla): se normaliza por z-score sobre el AOI y se acota a $\pm 2\sigma \rightarrow$ cresta 0.0 / valle 1.0.
- Existe una **barrera dura** de transitabilidad: pendiente (Horn) por encima del umbral $\rightarrow$ celda intransitable (inf $\rightarrow$ nan), independiente del coste. El umbral es **70 % por defecto ($\approx 35^\circ$ de inclinación) y editable desde el Paso 2** de la interfaz; el valor usado queda grabado como etiqueta del GeoTIFF de la capa TPI, que se regenera si cambia.
- El peso por perfil (en `perfiles.yaml`) modula el énfasis; el valor base de cada capa ya refleja la severidad oficial del condicionante.

Pesos por perfil — el mecanismo de diferenciación

Sobre las capas ya estandarizadas, cada **perfil** aplica un vector de pesos distinto. Misma base de coste + pesos distintos $\Rightarrow$ superficies distintas $\Rightarrow$ **rutas diferenciadas**: aquí es donde nacen los 4 trazados alternativos. Los perfiles son **configuración externa** (`perfiles.yaml`), no código: recalibrar es editar el YAML y re-ejecutar; también se editan en caliente desde la interfaz (sliders del Paso 2) sin tocar el archivo.

CAPA DE COSTE	CORTO	AMBIENTAL	RELIEVE (TPI)	EQUILIBRIO
Longitud (distancia base)	1,00	0,10	0,10	0,05
TPI (relieve)	0,13	0,10	1,00	0,12
Zonas protegidas (RN2000)	0,13	1,00	0,05	0,25
Zonas inundables (SNCZI)	0,07	0,50	0,05	0,12
Cruces (viario · ríos · ffcc)	0,20	0,50	0,05	0,16
Expropiación (catastro)	0,13	0,10	0,05	0,21
Geotecnia (litología)	0,07	0,80	0,10	0,09

- **corto · ambiental · relieve** usan pesos **relativos**: la capa que da nombre al perfil pesa 1,00 (dominante) y el resto en proporción — cada uno persigue un corredor claramente sesgado hacia un criterio.

- **equilibrio** (recomendado) usa pesos **normalizados que suman 1,00**, mapeando las bandas oficiales de Enagás (anexo II EV-500): medioambiente 30–40 % → protegida + inundable; urbanístico 15–25 % → expropiación; geotecnia 15–25 % → geotecnia + TPI; construcción 10–20 % → cruces; resto → longitud.
- Las siete capas llevan peso > 0 en los cuatro perfiles: ninguna alternativa ignora por completo un criterio, solo cambia su énfasis.

08 Estado del código y stack tecnológico

Stack

- **Lenguaje:** Python 3.10 (fijado en `.python-version`; entorno oficial proyecto/`.venv` creado por `setup.ps1/setup.sh`). Código en inglés, docs en español.
- **Geoespacial:** rasterio · geopandas · shapely · pyproj · numpy.
- **Descarga:** requests / urllib · osmnx-Overpass (WFS/WCS/OGC API).
- **UI y mapas:** Streamlit · folium · streamlit-folium.
- **Algoritmo LCP:** Dijkstra 8-conexo isótropo con heap (implementación propia).
- **CRS de trabajo:** ETRS89 / UTM 30N (EPSG:25830); vista en WGS-84.
- **Formatos:** GeoTIFF (raster) · GeoPackage (vector) · YAML (config) · JSON (manifiesto).